TALLER: PRUEBAS UNITARIAS I (GUÍA DEL APRENDIZ)

Aseguramiento de Calidad de Software y Lógica Computacional

Programa de Formación: Tecnología en Análisis y Desarrollo de Software (ADSO)

Competencia: Implementar la solución de software de acuerdo con los requisitos y el diseño (Fase de Evaluación / Pruebas)

Enfoque pedagógico: Patrón AAA (*Arrange - Act - Assert*), Análisis de Valores Límite (BVA), Partición de Equivalencia (EP) y Manejo de Excepciones.

Instrucciones para el Aprendiz

- **Metodología de trabajo:**
    1. Esta guía contiene **12 ejercicios prácticos** estructurados en 3 partes: el enunciado del problema, la implementación del código fuente que debe ser probado y los criterios de diseño para construir la suite de pruebas unitarias.
    2. Su objetivo es implementar las pruebas correspondientes aplicando los conceptos vistos en clase y la tabla maestra de aserciones.
- **Estándar de desarrollo de pruebas:** Todas las pruebas unitarias deben seguir de forma estricta las tres fases del patrón **AAA**:
    - **Arrange (Preparar):** Declaración de variables, datos de entrada y definición del oráculo (*valor esperado*).
    - **Act (Actuar):** Invocación de la función bajo prueba capturando el resultado.
    - **Assert (Verificar):** Aserción determinista contra el valor exacto esperado (sin desigualdades genéricas).
- **Framework de ejecución base:** `pytest` (Python 3.10+).
- **Condiciones de Entrega del Taller:**
    - La evidencia del taller **únicamente será recibida mediante el enlace (URL) al repositorio del aprendiz** (GitHub, GitLab, Gitea o Azure Repos).
    - El repositorio debe contener la historia de commits con los avances de la actividad, la estructura de carpetas indicada y el archivo de requerimientos (`requirements.txt`). No se recibirán archivos comprimidos (.zip / .rar).

Guía Técnica de Configuración del Entorno y Estructura del Proyecto

Para configurar su proyecto y ejecutar sus pruebas sin necesidad de consultar fuentes externas, ejecute los siguientes comandos en la terminal integrada de su editor (VS Code, PyCharm o terminal del sistema operativo):

1. Estructura de Directorios Obligatoria

El proyecto debe mantener de manera estricta la siguiente organización:

```
None
taller-pruebas-unitarias/
├── .gitignore
├── requirements.txt
├── README.md
└── src/
    ├── __init__.py
    ├── ejercicio_01.py
    ├── ejercicio_02.py
    ├── ...
    ├── ejercicio_12.py
    └── tests/
        ├── __init__.py
        ├── test_ejercicio_01.py
        ├── test_ejercicio_02.py
        ├── ...
        └── test_ejercicio_12.py
```

- Cada función desarrollada en la **Parte 2** debe ubicarse en su archivo independiente dentro de `src/` (por ejemplo: `src/ejercicio_01.py`).
- Cada archivo de prueba unitaria que usted diseñe a partir de la **Parte 3** debe ubicarse dentro de `src/tests/` con el prefijo obligatorio `test_` (por ejemplo: `src/tests/test_ejercicio_01.py`).

---

2. Comandos de Inicialización y Creación de Carpetas

Abra una terminal en la carpeta donde almacenará sus proyectos y ejecute:

```
Shell
# Crear directorio principal del taller y acceder a él
mkdir taller-pruebas-unitarias
```

```
cd taller-pruebas-unitarias

# Crear las carpetas de código fuente y pruebas
mkdir -p src/tests

# Crear archivos de inicialización de paquete Python
touch src/__init__.py src/tests/__init__.py
```

(En Windows PowerShell, si el comando `touch` no está disponible, use: `New-Item -ItemType File src/__init__.py, src/tests/__init__.py`)

---

3. Creación y Activación del Entorno Virtual de Python

El entorno virtual aísla las dependencias del proyecto evitando conflictos globales:

- **En Windows (CMD o PowerShell):**

```powershell
# Crear el entorno virtual llamado 'venv'
python -m venv venv

# Activar el entorno virtual (PowerShell)
.\venv\Scripts\Activate.ps1

# Si PowerShell bloquea scripts por directiva de ejecución, ejecute antes:
# Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process

# O si utiliza el Símbolo del Sistema tradicional (CMD):
# .\venv\Scripts\activate.bat
```

- **En Linux o macOS (Bash / Zsh):**

```shell
# Crear el entorno virtual llamado 'venv'
python3 -m venv venv

# Activar el entorno virtual
source venv/bin/activate
```

(Sabrá que el entorno está activo porque verá el prefijo `(venv)` al inicio de su línea de comandos).

---

4. Instalación de Dependencias y Congelamiento

Con el entorno virtual activo:

```shell
# Actualizar el gestor de paquetes pip
pip install --upgrade pip

# Instalar pytest
pip install pytest

# Guardar la versión exacta instalada en requirements.txt
pip freeze > requirements.txt
```

---

5. Configuración del Archivo `.gitignore`

Cree un archivo llamado `.gitignore` en la raíz del proyecto para evitar subir archivos temporales y el entorno virtual al repositorio:

```shell
# Crear y poblar el archivo .gitignore
cat <<EOT >> .gitignore
venv/
__pycache__/
*.pyc
.pytest_cache/
.DS_Store
EOT
```

(En Windows PowerShell puede crear el archivo `.gitignore` y escribir dentro de él las siguientes líneas: `venv/`, `__pycache__/`, `*.pyc`, `.pytest_cache/`).

---

6. Comandos para Ejecución de Pruebas Unitarias

Para que Python y `pytest` reconozcan la carpeta `src` en el `PYTHONPATH` al importar los módulos, ejecute las pruebas desde la raíz del proyecto utilizando cualquiera de las siguientes opciones:

Shell

```
# Opción recomendada: ejecuta pytest como módulo reconociendo la raíz del proyecto
python -m pytest -v

# Ejecutar únicamente las pruebas de un ejercicio específico (ejemplo ejercicio 01)
python -m pytest -v src/tests/test_ejercicio_01.py

# Ejecutar mostrando salidas por consola print detalladas (-s)
python -m pytest -v -s
```

---

7. Comandos de Control de Versiones (Git) y Publicación del Repositorio

Una vez verificado el funcionamiento local, vincule su carpeta con su repositorio en GitHub/GitLab:

Shell

```
# Inicializar el repositorio Git local
git init

# Cambiar la rama principal a 'main'
git branch -M main

# Añadir todos los archivos al área de preparación (staging)
git add .

# Crear el primer commit con mensaje descriptivo
git commit -m "feat: estructura inicial del proyecto y configuracion de entorno de pruebas"

# Vincular con su repositorio remoto (reemplace la URL por la de su repositorio)
git remote add origin https://github.com/SU_USUARIO/taller-pruebas-unitarias.git

# Enviar los cambios a la rama principal remota
git push -u origin main
```

(Para entregas posteriores tras completar los ejercicios, repita: `git add .`, `git commit -m "feat: implementacion ejercicios y pruebas unitarias"` y `git push`).

Ejercicio 01: Suma Segura de Dos Números

Parte 1: Enunciado de la solución

Cree una función llamada `sumar(a, b)` en `src/ejercicio_01.py` que reciba dos parámetros numéricos (enteros o flotantes) y devuelva su suma. La función debe validar que ambos argumentos sean estrictamente numéricos (`int` o `float`), pero rechazando booleanos explícitamente (dado que en Python `isinstance(True, int)` es verdadero). Si recibe un argumento inválido, debe lanzar un `TypeError`.

Parte 2: Código que resuelve la solución del enunciado

Python

```
def sumar(a, b):
    # En Python, bool es subclase de int, por lo que se valida explícitamente
    if type(a) not in (int, float) or type(b) not in (int, float):
        raise TypeError("Ambos argumentos deben ser estrictamente numéricos (int o float)")
    return a + b
```

Parte 3: Enunciado para el diseño de la prueba unitaria

En `src/tests/test_ejercicio_01.py`, diseñe una suite de pruebas unitarias que verifique:

1. **Happy Path:** Suma de dos enteros positivos (2 + 3 = 5).
2. **Suma con negativos:** Suma de un entero positivo y un negativo (10 + (-4) = 6).
3. **Manejo de tipos inválidos:** Lanzamiento de `TypeError` cuando uno de los argumentos es un string (`"5"`).
4. **Manejo de booleanos:** Lanzamiento de `TypeError` cuando se envía un booleano (`True`).

Ejercicio 02: Número Par o Impar

Parte 1: Enunciado de la solución

Cree una función llamada `es_par(numero)` en `src/ejercicio_02.py` que determine si un número entero es par. Debe retornar `True` si es par y `False` si es impar. Si el parámetro recibido no es un número entero exacto, debe arrojar una excepción `TypeError`.

Parte 2: Código que resuelve la solución del enunciado

```python
def es_par(numero):
    if type(numero) is not int:
        raise TypeError("El parámetro debe ser un número entero")
    return numero % 2 == 0
```

Parte 3: Enunciado para el diseño de la prueba unitaria

En `src/tests/test_ejercicio_02.py`, diseñe pruebas unitarias estructuradas para validar:

1. Caso positivo con número par positivo (ej. `8`).
2. Caso negativo con número impar positivo (ej. `7`).
3. Comportamiento en el valor neutro `0` (matemáticamente par).
4. Comportamiento con enteros negativos pares e impares (ej. `-4` y `-9`).
5. Rechazo con `TypeError` si se ingresa un número decimal (`4.0`).

Ejercicio 03: Conversión de Celsius a Fahrenheit

Parte 1: Enunciado de la solución

Cree una función llamada `celsius_a_fahrenheit(celsius)` en `src/ejercicio_03.py` que transforme una temperatura en grados Celsius a Fahrenheit utilizando la fórmula: `F = (celsius * 1.8) + 32`

La función debe rechazar temperaturas por debajo del cero absoluto (-273.15 °C) lanzando un `ValueError`.

Parte 2: Código que resuelve la solución del enunciado

```python
def celsius_a_fahrenheit(celsius):
    if type(celsius) not in (int, float):
        raise TypeError("La temperatura debe ser un número")
    if celsius < -273.15:
        raise ValueError("Temperatura por debajo del cero absoluto (-273.15 °C)")
    return (celsius * 1.8) + 32
```

Parte 3: Enunciado para el diseño de la prueba unitaria

En `src/tests/test_ejercicio_03.py`, diseñe pruebas unitarias para evaluar:

1. Punto de congelación del agua: 0 °C = 32.0 °F.
2. Punto de ebullición del agua: 100 °C = 212.0 °F.
3. Punto de coincidencia de escalas: -40 °C = -40.0 °F.
4. Precisión decimal con `pytest.approx`: 37 °C = 98.6 °F.
5. Límite inferior no permitido: -273.16 °C debe lanzar `ValueError`.

Ejercicio 04: Cálculo de Descuento Comercial

Parte 1: Enunciado de la solución

Cree una función llamada `aplicar_descuento(precio, porcentaje_descuento)` en `src/ejercicio_04.py` que devuelva el precio final a pagar tras aplicar un porcentaje de descuento.

Reglas de negocio:

- El precio no puede ser negativo (`precio >= 0`).
- El porcentaje de descuento debe estar estrictamente en el rango de 0 a 100 inclusive (`0 <= porcentaje_descuento <= 100`).
- Si alguna regla no se cumple, debe arrojar `ValueError`.

Parte 2: Código que resuelve la solución del enunciado

```python
def aplicar_descuento(precio, porcentaje_descuento):
    if not (isinstance(precio, (int, float)) and isinstance(porcentaje_descuento, (int, float))):
        raise TypeError("Los valores deben ser numéricos")
```

```
    if isinstance(precio, bool) or isinstance(porcentaje_descuento, bool):
        raise TypeError("Los valores no pueden ser booleanos")
    if precio < 0:
        raise ValueError("El precio no puede ser negativo")
    if porcentaje_descuento < 0 or porcentaje_descuento > 100:
        raise ValueError("El descuento debe estar entre 0 y 100")

    descuento = precio * (porcentaje_descuento / 100)
    return precio - descuento
```

Parte 3: Enunciado para el diseño de la prueba unitaria

En `src/tests/test_ejercicio_04.py`, construya casos de prueba para validar:

1. Caso general: Precio de 100000 con 20% de descuento -> 80000.0.
2. Límite inferior de descuento: 0% de descuento sobre 50000 -> 50000.0.
3. Límite superior de descuento: 100% de descuento sobre 75000 -> 0.0.
4. Excepción de descuento menor a cero: Porcentaje -1% -> `ValueError`.
5. Excepción de descuento mayor a cien: Porcentaje 101% -> `ValueError`.

Ejercicio 05: Verificación de Mayoría de Edad

Parte 1: Enunciado de la solución

Cree una función `es_mayor_de_edad(edad)` en `src/ejercicio_05.py` que reciba la edad en años cumplidos (entero) y devuelva:

- `True` si la persona tiene 18 años o más.
- `False` si la persona tiene menos de 18 años.
- Debe lanzar `ValueError` si la edad es menor a 0 o mayor a 125.
- Debe lanzar `TypeError` si la edad no es un entero.

Parte 2: Código que resuelve la solución del enunciado

Python

```
def es_mayor_de_edad(edad):
    if type(edad) is not int:
        raise TypeError("La edad debe ser un número entero")
    if edad < 0 or edad > 125:
        raise ValueError("La edad ingresada está fuera del rango biológico
admisible [0 - 125]")
```

```
    return edad >= 18
```

Parte 3: Enunciado para el diseño de la prueba unitaria

En `src/tests/test_ejercicio_05.py`, implemente pruebas aplicando la técnica de **Análisis de Valores Límite (BVA)** en la frontera crítica de los 18 años:

1. Inmediatamente inferior a la frontera: 17 años -> `False`.
2. Frontera exacta de mayoría de edad: 18 años -> `True`.
3. Inmediatamente superior a la frontera: 19 años -> `True`.
4. Límite biológico inferior inválido: -1 -> `ValueError`.
5. Límite biológico superior inválido: 126 -> `ValueError`.

Ejercicio 06: Longitud de Contraseña

Parte 1: Enunciado de la solución

Cree una función `validar_longitud_password(password)` en `src/ejercicio_06.py` que determine si una contraseña cumple con el criterio de seguridad de longitud mínima y máxima.

- Retorna `True` si la contraseña tiene entre 8 y 20 caracteres inclusive (`8 <= longitud <= 20`).
- Retorna `False` si tiene menos de 8 o más de 20 caracteres.
- Lanza `TypeError` si el argumento no es una cadena de texto (`str`).

Parte 2: Código que resuelve la solución del enunciado

Python

```
def validar_longitud_password(password):
    if not isinstance(password, str):
        raise TypeError("El password debe ser una cadena de caracteres")
    return 8 <= len(password) <= 20
```

Parte 3: Enunciado para el diseño de la prueba unitaria

En `src/tests/test_ejercicio_06.py`, diseñe pruebas enfocadas en los límites de caracteres de la cadena:

1. Longitud 7 caracteres (borde inválido inferior) -> `False`.
2. Longitud 8 caracteres (borde válido inferior) -> `True`.
3. Longitud 20 caracteres (borde válido superior) -> `True`.
4. Longitud 21 caracteres (borde inválido superior) -> `False`.
5. Paso de valor nulo (`None`) o numérico -> `TypeError`.

Ejercicio 07: Cálculo del Factorial

Parte 1: Enunciado de la solución

Cree una función `factorial(n)` en `src/ejercicio_07.py` que calcule el factorial de un número entero no negativo (n!).

- Recuerde que: 0! = 1 y 1! = 1.
- Para n > 1: n! = n * (n - 1) * ... * 1.
- Si n < 0, debe lanzar `ValueError("El factorial no está definido para números negativos")`.
- Si n no es entero, debe lanzar `TypeError`.

Parte 2: Código que resuelve la solución del enunciado

Python

```python
def factorial(n):
    if type(n) is not int:
        raise TypeError("El argumento debe ser un número entero")
    if n < 0:
        raise ValueError("El factorial no está definido para números negativos")

    resultado = 1
    for i in range(2, n + 1):
        resultado *= i
    return resultado
```

Parte 3: Enunciado para el diseño de la prueba unitaria

En `src/tests/test_ejercicio_07.py`, diseñe pruebas que validen:

1. Caso base n = 0 -> 1.
2. Caso base n = 1 -> 1.
3. Caso típico n = 5 -> 120.
4. Caso con número mayor n = 7 -> 5040.
5. Validación de error cuando n = -3 -> lanza `ValueError`.

Ejercicio 08: Obtener el Número Mayor de una Lista

Parte 1: Enunciado de la solución

Cree una función `obtener_mayor(lista_numeros)` en `src/ejercicio_08.py` que reciba una lista de números y devuelva el número con el valor más alto de la secuencia sin usar la función nativa `max()`.

- Si la lista está vacía, debe lanzar un `ValueError("La lista no puede estar vacía")`.
- Si el argumento no es una lista o contiene elementos no numéricos, debe lanzar `TypeError`.

Parte 2: Código que resuelve la solución del enunciado

Python

```python
def obtener_mayor(lista_numeros):
    if not isinstance(lista_numeros, list):
        raise TypeError("El argumento debe ser una lista")
    if len(lista_numeros) == 0:
        raise ValueError("La lista no puede estar vacía")

    mayor = lista_numeros[0]
    for elemento in lista_numeros:
        if type(elemento) not in (int, float):
            raise TypeError("Todos los elementos de la lista deben ser números")
        if elemento > mayor:
            mayor = elemento

    return mayor
```

Parte 3: Enunciado para el diseño de la prueba unitaria

En `src/tests/test_ejercicio_08.py`, construya los siguientes casos de prueba:

1. Lista con enteros positivos desordenados: `[4, 12, 8, 3]` -> 12.
2. Lista que contiene números negativos: `[-15, -3, -8, -20]` -> -3.
3. Lista de un único elemento: `[42]` -> 42.
4. Manejo de error en lista vacía: `[]` -> `ValueError`.
5. Manejo de error con elementos corruptos: `[1, 2, "tres"]` -> `TypeError`.

Ejercicio 09: Verificación de Palíndromos

Parte 1: Enunciado de la solución

Cree una función `es_palindromo(cadena)` en `src/ejercicio_09.py` que determine si una palabra o frase se lee igual hacia adelante que hacia atrás.

- La función debe ignorar mayúsculas/minúsculas y espacios en blanco.
- Debe devolver `True` si es palíndromo y `False` si no lo es.
- Si el texto recibido está vacío o contiene solo espacios, debe retornar `False`.
- Si el tipo no es `str`, debe lanzar `TypeError`.

Parte 2: Código que resuelve la solución del enunciado

Python

```python
def es_palindromo(cadena):
    if not isinstance(cadena, str):
        raise TypeError("El parámetro debe ser una cadena de texto")

    # Limpieza: eliminar espacios y pasar a minúsculas
    limpia = "".join(cadena.split()).lower()

    if len(limpia) == 0:
        return False

    return limpia == limpia[::-1]
```

Parte 3: Enunciado para el diseño de la prueba unitaria

En `src/tests/test_ejercicio_09.py`, plantee pruebas unitarias que verifiquen:

1. Palabra palíndroma simple: `"radar"` -> `True`.
2. Frase con espacios y mayúsculas: `"Anita lava la tina"` -> `True`.
3. Palabra no palíndroma: `"software"` -> `False`.
4. Cadena vacía o con solo espacios: `"   "` -> `False`.
5. Rechazo de tipos numéricos: `1001` -> `TypeError`.

Ejercicio 10: Clasificación de Calificaciones SENA

Parte 1: Enunciado de la solución

Cree una función `clasificar_nota(nota)` en `src/ejercicio_10.py` que devuelva la descripción cualitativa del desempeño académico sobre una escala de 0.0 a 5.0:

- Si 4.5 <= nota <= 5.0: Retorna `"Excelente"`
- Si 3.5 <= nota < 4.5: Retorna `"Bueno"`
- Si 3.0 <= nota < 3.5: Retorna `"Aceptable"`
- Si 0.0 <= nota < 3.0: Retorna `"Deficiente"`
- Cualquier nota fuera de la escala (nota < 0.0 o nota > 5.0) debe arrojar `ValueError("Nota fuera de la escala [0.0 - 5.0]")`.

Parte 2: Código que resuelve la solución del enunciado

Python

```python
def clasificar_nota(nota):
    if type(nota) not in (int, float):
        raise TypeError("La nota debe ser un número flotante o entero")
    if nota < 0.0 or nota > 5.0:
        raise ValueError("Nota fuera de la escala [0.0 - 5.0]")

    if nota >= 4.5:
        return "Excelente"
    elif nota >= 3.5:
        return "Bueno"
    elif nota >= 3.0:
        return "Aceptable"
    else:
        return "Deficiente"
```

Parte 3: Enunciado para el diseño de la prueba unitaria

En `src/tests/test_ejercicio_10.py`, diseñe pruebas para las **particiones de equivalencia** y sus **fronteras**:

1. Límite superior máximo: 5.0 -> `"Excelente"`.
2. Borde inferior de "Excelente": 4.5 -> `"Excelente"`.
3. Borde superior de "Bueno": 4.4 -> `"Bueno"`.
4. Borde inferior de aprobación mínima: 3.0 -> `"Aceptable"`.
5. Borde inmediatamente reprobatorio: 2.9 -> `"Deficiente"`.
6. Límite inferior absoluto: 0.0 -> `"Deficiente"`.
7. Valores fuera de escala: -0.1 y 5.1 -> lanzan `ValueError`.

Ejercicio 11: División Segura

Parte 1: Enunciado de la solución

Cree una función `division_segura(dividendo, divisor)` en `src/ejercicio_11.py` que efectúe la operación de división.

- Debe devolver el resultado como `float`.
- Si el `divisor` es igual a 0, la función debe lanzar explícitamente `ZeroDivisionError("No es posible dividir entre cero")`.
- Si alguno de los argumentos no es numérico, debe lanzar `TypeError`.

Parte 2: Código que resuelve la solución del enunciado

Python

```python
def division_segura(dividendo, divisor):
    if type(dividendo) not in (int, float) or type(divisor) not in (int, float):
        raise TypeError("Los operandos deben ser numéricos")
    if divisor == 0:
        raise ZeroDivisionError("No es posible dividir entre cero")
    return dividendo / divisor
```

Parte 3: Enunciado para el diseño de la prueba unitaria

En `src/tests/test_ejercicio_11.py`, implemente pruebas unitarias para validar:

1. División exacta: 10 / 2 = 5.0.
2. División periódica con verificación de precisión flotante: 10 / 3 = 3.3333... con `pytest.approx`.
3. Cero en el dividendo: 0 / 5 = 0.0.
4. División por cero en el divisor: 8 / 0 -> comprobar captura de `ZeroDivisionError`.

Ejercicio 12: Validación de Diccionario de Usuario

Parte 1: Enunciado de la solución

Cree una función `procesar_usuario(datos_usuario)` en `src/ejercicio_12.py` que reciba un diccionario con información de registro y valide que contenga obligatoriamente las siguientes claves:

- `"id"` (entero positivo mayor a 0)
- `"nombre"` (cadena no vacía)
- `"activo"` (booleano)

Si el diccionario cumple las condiciones, la función devuelve una copia del diccionario agregando el campo `"estado_registro": "VALIDADO"`. Si falta alguna clave requerida o los tipos de datos son erróneos, debe lanzar `ValueError` con un mensaje explicativo.

Parte 2: Código que resuelve la solución del enunciado

Python

```
def procesar_usuario(datos_usuario):
    if not isinstance(datos_usuario, dict):
        raise TypeError("El parámetro debe ser un diccionario")

    claves_requeridas = {"id", "nombre", "activo"}
    if not claves_requeridas.issubset(datos_usuario.keys()):
        faltantes = claves_requeridas - datos_usuario.keys()
        raise ValueError(f"Faltan campos requeridos en el registro: {faltantes}")

    if type(datos_usuario["id"]) is not int or datos_usuario["id"] <= 0:
        raise ValueError("El id debe ser un entero positivo mayor a cero")

    if not isinstance(datos_usuario["nombre"], str) or
len(datos_usuario["nombre"].strip()) == 0:
        raise ValueError("El nombre debe ser una cadena no vacía")

    if type(datos_usuario["activo"]) is not bool:
        raise ValueError("El campo activo debe ser un valor booleano")

    nuevo_registro = datos_usuario.copy()
    nuevo_registro["estado_registro"] = "VALIDADO"
    return nuevo_registro
```

Parte 3: Enunciado para el diseño de la prueba unitaria

En `src/tests/test_ejercicio_12.py`, diseñe pruebas que validen:

1. Caso exitoso: Diccionario con todos los campos correctos -> retorna el diccionario con `"estado_registro": "VALIDADO"` y conserva las claves originales.
2. Clave faltante: Diccionario sin la clave `"activo"` -> lanza `ValueError`.
3. Campo `"id"` inválido (cero o negativo) -> lanza `ValueError`.
4. Campo `"nombre"` vacío o solo con espacios -> lanza `ValueError`.
5. Paso de argumento que no es diccionario (ej. `list`) -> lanza `TypeError`.

TABLA MAESTRA COMPARATIVA DE ASSERTS Y VERIFICACIONES EN PYTHON

Referencia Rápida para el Aprendiz: `pytest` vs. `unittest`

Esta tabla sirve como catálogo de consulta para resolver los casos de prueba de este taller utilizando los dos entornos estándar de Python: la sintaxis declarativa de `pytest` y los métodos del módulo nativo estándar `unittest`:

Intención de Prueba	Sintaxis PyTest (`pytest`)	Módulo Estándar (`unittest`)
Igualdad de valor simple	`assert actual == esperado`	`self.assertEqual(a, b)`
Igualdad de diccionarios / estructuras	`assert actual == esperado_dict`	`self.assertDictEqual(a, b)`
Desigualdad	`assert actual != no_esperado`	`self.assertNotEqual(a, b)`
Booleano Verdadero (`True`)	`assert actual is True`	`self.assertTrue(actual)`
Booleano Falso (`False`)	`assert actual is False`	`self.assertFalse(actual)`
Valor Nulo (`None`)	`assert actual is None`	`self.assertIsNone(actual)`
Veracidad / Falsedad general	`assert actual` / `assert not actual`	`self.assertTrue(bool(x))`
Aproximación decimal (Flotantes)	`assert actual == pytest.approx(3.14, rel=1e-3)`	`self.assertAlmostEqual(a, b, places=3)`
Comparación de orden numérico	`assert a > b` / `assert a <= b`	`self.assertGreater(a, b)` / `self.assertLessEqual(a, b)`
Verificar tipo de dato	`assert isinstance(x, int)` / `assert type(x) is int`	`self.assertIsInstance(x, int)`

Intención de Prueba	Sintaxis PyTest (`pytest`)	Módulo Estándar (`unittest`)
Elemento dentro de colección/lista	`assert item in lista`	`self.assertIn(item, lista)`
Clave en diccionario	`assert "clave" in diccionario`	`self.assertIn("clave", d)`
Longitud de secuencia o texto	`assert len(cadena) == 8`	`self.assertEqual(len(x), 8)`
Coincidencia de texto (Expresión Regular)	`assert re.search(r"patron", texto)`	`self.assertRegex(texto, r"patron")`
Esperar lanzamiento de Excepción / Error	`with pytest.raises(ValueError): func()`	`with self.assertRaises(ValueError): func()`
Esperar Excepción con mensaje específico	`with pytest.raises(ValueError, match="msg"): func()`	`with self.assertRaisesRegex(ValueError, "msg"): func()`
